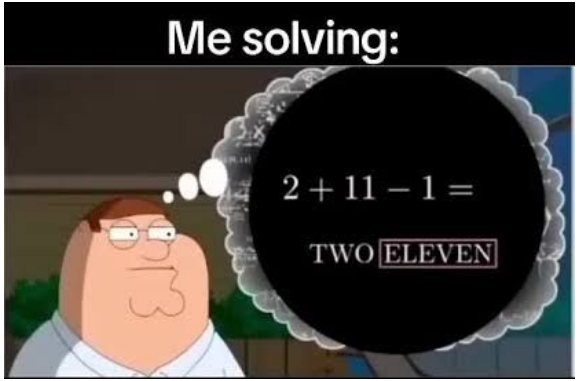
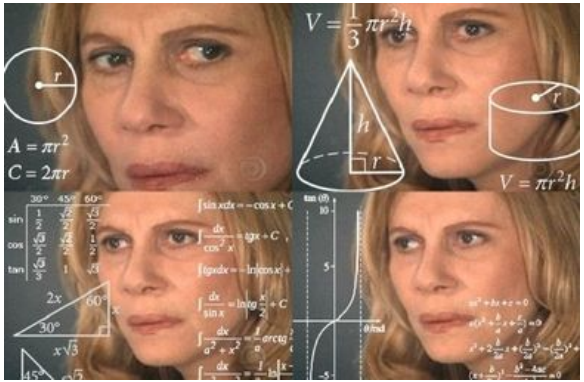


תכנות תחרותי

הרצאה 6



שאלה פשוטה

- למלך ארתור יש n<10^5 אבירים. הוא מעוניין להושיב אותם סביב השולחן העגול שלו.
- כמה דרכים שונות יש לארתור להושיב את האבירים סביב השולחן?
- שתי דרכים נקראות זהות אם יש ביניהן סימטריה סיבובית

שאלה פשוטה

- כמובן שהתשובה היא $(n-1)!$, פשוט
- מה קורה כאשר n=50?

Calculator for find the factorial

49 ! = 608281864034267560872252163321295376887552831379210240000000000

modulo

Calculator for find the factorial

49 ! = 608281864034267560872252163321295376887552831379210240000000000

- בעיה! long long מכיל עד כ- 10^{18} . יהיה לנו overflow!
- הדרך שלנו לפתור את זה היא להשתמש במודולו

modulo

- יכול לראות שיצא לכם להיתקל בעבר במספרים 998244353 , $10^9 + 7$
- מה מיוחד במספרים האלה?
  - הם גדולים
  - הם ראשוניים

modulo

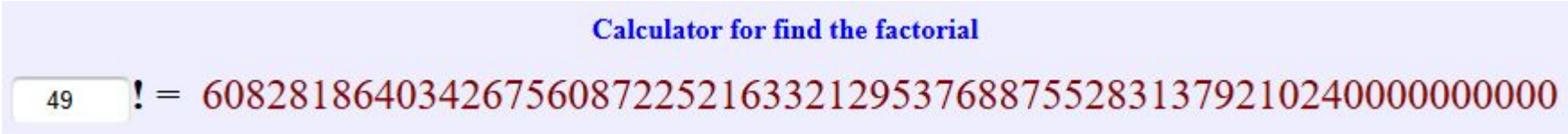
- המספר צריך להיות גדול כדי להקטין את הסיכוי ששתי תוצאות שונות יהיו שוות תחת המודולו בטעות. למשל המספרים 3,8,13,18... כולם שווים תחת מודולו 5.
- את הסיבה שהמספר ראשוני אנחנו נראה עוד מעט

שאלה פשוטה

- למלך ארתור יש n<10^5 אבירים. הוא מעוניין להושיב אותם סביב השולחן העגול שלו.
- כמה דרכים שונות יש לארתור להושיב את האבירים סביב השולחן?

שאלה פשוטה

- כמובן שהתשובה היא $(n-1)!$, פשוט
- מה קורה כאשר n=50?



- אין בעיה, נדפיס תחת מודולו

```cpp
const int MOD = 1000000007;

void solve() {
    int n; cin >> n;

    int ans = 1;
    rep(i,1,n) ans = (ans*i)%MOD;
    cout << ans;
}
```

```
50

726372166
Process finished with exit code 0
```

שאלה עדיין פשוטה

- למלך ארתור יש n<10^5 אבירים. הוא מעוניין להושיב אותם סביב השולחן העגול שלו.
- כל יום ארתור רוצה להחליף את הסידור של האבירים בשולחן לסדר שעוד לא היה בעבר, כמה שבועות ארתור יוכל לעשות את זה?
- שתי דרכים נקראות זהות אם יש ביניהן סימטריה סיבובית

שאלה עדיין פשוטה

- לא הרבה יותר מסובך, $\frac{(n-1)!}{7}$
- אבל רגע, איך עושים חילוק תחת מודולו?

מה אנחנו כן יודעים?

- חיבור: `ans = (a + b) % MOD;`
- חיסור: `ans = (a - b + MOD) % MOD;`
- כפל: `ans = (a * b) % MOD;`
- מה לגבי חילוק?

חילוק תחת מודולו

- איזו תכונה חילוק צריך לקיים?

חילוק תחת מודולו

- איזו תכונה חילוק צריך לקיים?

$$a/b = c \Leftrightarrow c * b = a$$

The floor is a proof

הלמה הקטנה של פירמה

- בשנת 1640 המתמטיקאי פייר דה פרמה הציג לראשונה את הלמה לפיה לכל מספר ראשוני p ולכל מספר שלם a

$$a^p \equiv a \ (mod\ p)$$

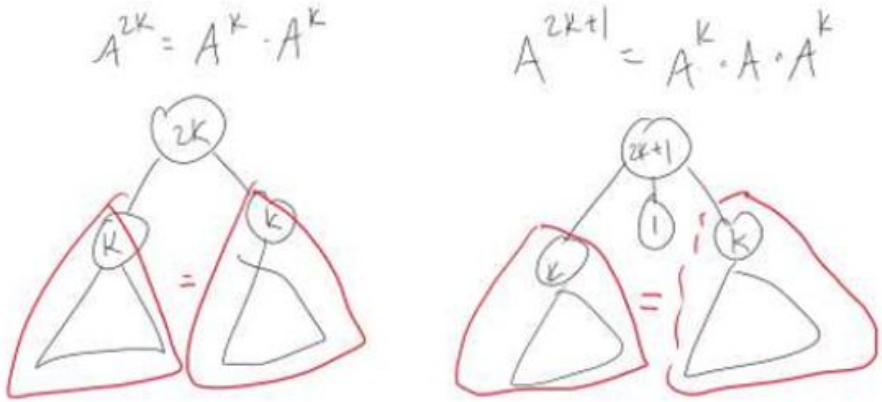
- לכן עבור מספר a זר ל-p (לא מתחלק ב-p) נקבל

$$a^{p-2} \equiv a^{-1} \ (mod\ p)$$

- כלומר נוכל לבצע חילוק על ידי כפל בהופכי בעזרת חזקה!

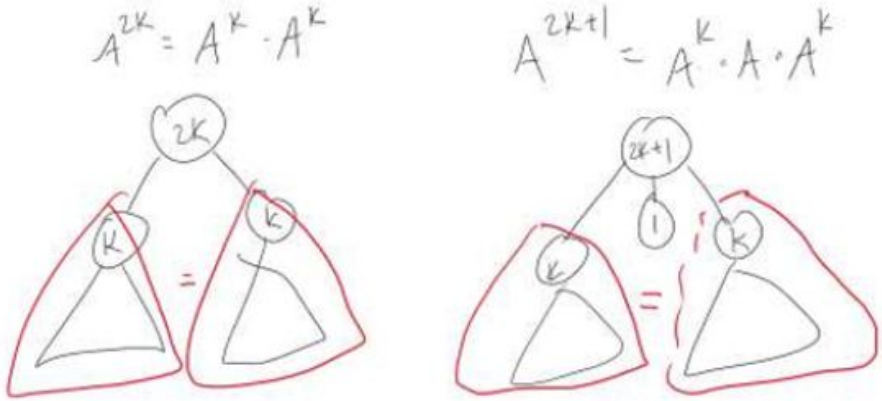
חזקה תחת מודולו

- הבנו שכדי לחלק אנחנו צריכים להעלות בחזקה, אבל עולה השאלה איך אפשר להעלות בחזקה? מה הסיבוכיות של פעולה כזאת?
- כמובן שאפשר בזמן לינארי אבל אנחנו רוצים יותר טוב



חזקה תחת מודולו

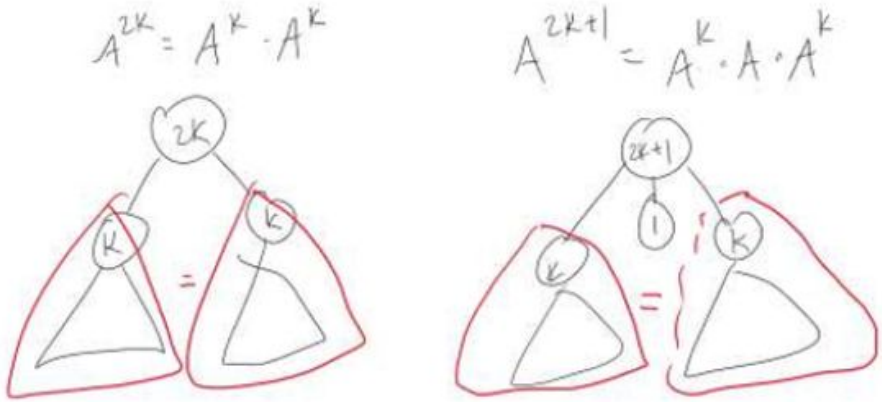
- נראה כמו תכנון דינאמי!
- נשים לב שאין טעם להיכנס לבן הימני של העץ אם כבר חישבנו את השמאלי
- סיבוכיות זמן לוגריתמית מעולה




```cpp
const int md = 1e9+7;
int pw(int a, int b){
    a %= md;
    if(b < 2) return b ? a : 1;
    int a_sqr = pw(a,  b: b/2);
    return (((b&1) ? a : 1) * a_sqr % md) * a_sqr % md;
}
```

חזקה תחת מודולו

- אפשר גם בלי רקורסיה!
- נפרק את b לייצוג הבינארי שלו




```cpp
const int md = 1e9+7;
int pw(int a, int b){
    int res = 1;
    while(b > 0){
        if(b & 1) res = res * a % md;
        b /= 2;
        a = a * a % md;
    } return res;
}
```

עצרת תחת מודולו

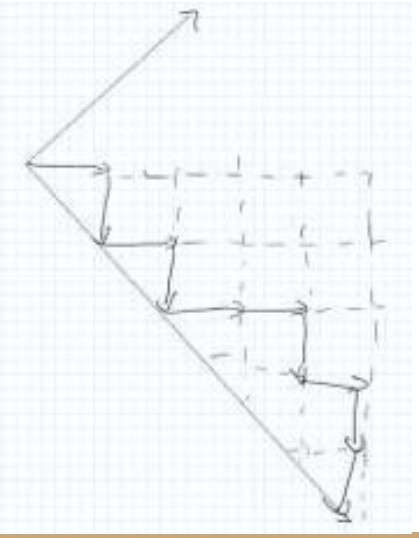
- עצרת תחת מודולו נבצע פשוט בזמן לינארי. בדרך כלל נהוג לעשות preprocessing ולחשב את כל העצרות לפני שקולטים בכלל קלט.

מה לגבי בינום?

- אם ניתן לעשות עצרת ולחלק, כמובן שנוכל לעשות גם בינום
- $\binom{n}{k} = \frac{n!}{(n-k)!\cdot k!}$

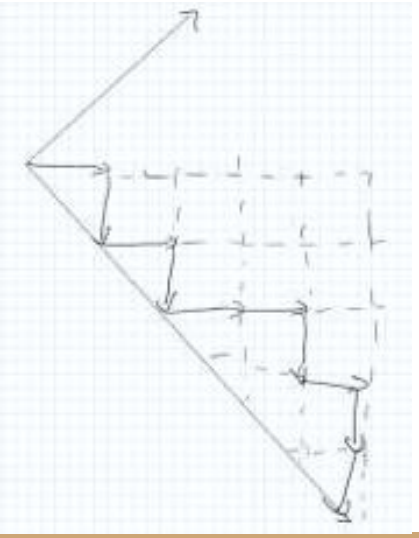
שאלה בקומבינטוריקה

- בהינתן טבלה מגודל NxN, בצעד מסויים מותר לזוז ימינה או למטה בטבלה.
- אתם מתחילים בפינה השמאלית עליונה.
- כמה דרכים שונות יש להגיע לפינה הימנית תחתונה מבלי לחצות את האלכסון הראשי?



שאלה פשוטה יותר

- בהינתן טבלה מגודל NxN, בצעד מסויים מותר לזוז ימינה או למטה בטבלה.
- אתם מתחילים בפינה השמאלית עליונה.
- כמה דרכים שונות יש להגיע לפינה הימנית תחתונה?



- כמובן שהתשובה $\binom{2n}{n}$. יש סך הכל 2n צעדים מתוכם n צעדים למטה.

שאלה בקומבינטוריקה

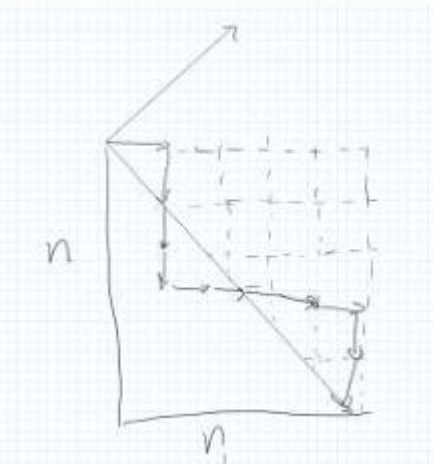
- בהינתן טבלה מגודל NxN, בצעד מסויים מותר לזוז ימינה או למטה בטבלה.
- אתם מתחילים בפינה השמאלית עליונה.
- כמה דרכים שונות יש להגיע לפינה הימנית תחתונה מבלי לחצות את האלכסון הראשי?

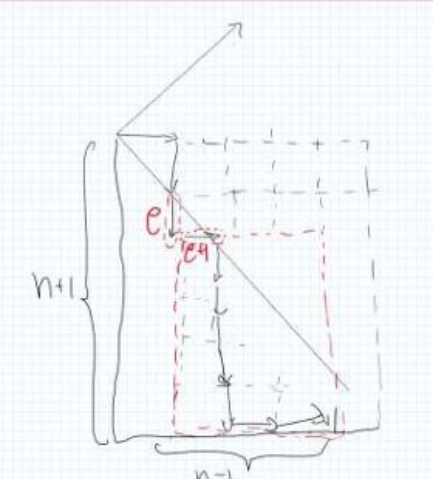
- מצאנו כבר כמה דרכים יש אם מותר לנו לעבור את האלכסון הראשי,
- נשאר לנו רק למצוא כמה דרכים בהן אנחנו כן חוצים את האלכסון הראשי ולחסר

שאלה בקומבינטוריקה

- בהינתן טבלה מגודל NxN, בצעד מסויים מותר לזוז ימינה או למטה בטבלה.
- אתם מתחילים בפינה השמאלית עליונה.
- כמה דרכים שונות יש להגיע לפינה הימנית תחתונה מבלי לחצות את האלכסון הראשי?

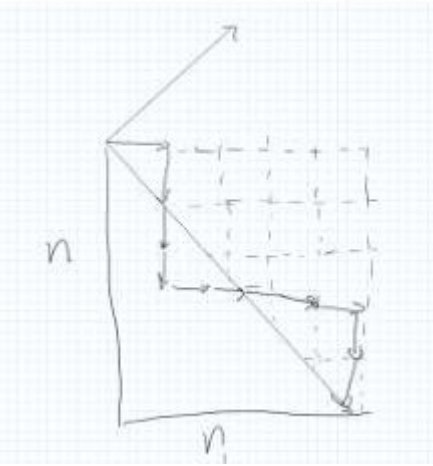
- מצאנו כבר כמה דרכים יש אם מותר לנו לעבור את האלכסון הראשי,
- נשאר לנו רק למצוא כמה דרכים בהן אנחנו כן חוצים את האלכסון הראשי ולחסר!

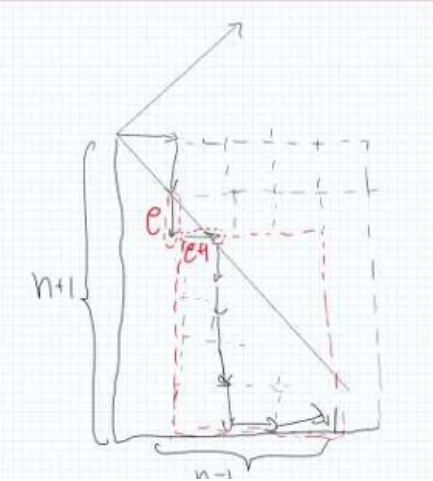




שאלה בקומבינטוריקה

- נסתכל על מסלול כזה שחוצה את האלכסון הראשי בצעד e.
- כלומר e הוא הפעם הראשונה שאנחנו יורדים מתחת לאלכסון הראשי.
- אם נהפוך את כל הצעדים שלנו מהצעד ה-e+1 והלאה (כלומר כל צעד ימינה יהפוך לצעד למטה וכל צעד למטה יהפוך לצעד ימינה) אנחנו תמיד נקבל מסלול חדש שהוא מסלול בטבלה מגודל $(N+1)x(N-1)$, בנוסף כמובן שכל מסלול בטבלה הזו יורד מתחת לאלכסון הראשי ולכן יש התאמה חד חד ערכית ועל.





שאלה בקומבינטוריקה

- קיבלנו שיש $\binom{2n}{n-1}$ מסלולים שיורדים מתחת לאלכסון הראשי בטבלה המקורית לכן יש $\binom{2n}{n} - \binom{2n}{n-1} = \frac{1}{n+1}\binom{2n}{n}$ מסלולים שונים בטבלה NxN שלא יורדים מתחת לאלכסון הראשי

מספרי קטלן

- המספר שעכשיו חישבנו נקרא מספר קטלן מסדר n: $C_n = \frac{1}{n+1}\binom{2n}{n}$
- המספרים האלה מופיעים בהרבה שאלות קומבינטוריקה אז חשוב להכיר אותם

- Balanced Parentheses
- Binary Tree Structures
- Binary Search Trees
- Polygon Triangulations
- Non-Crossing Matchings
- Dyck Paths
- Stack Permutations
- Non-Crossing Partitions

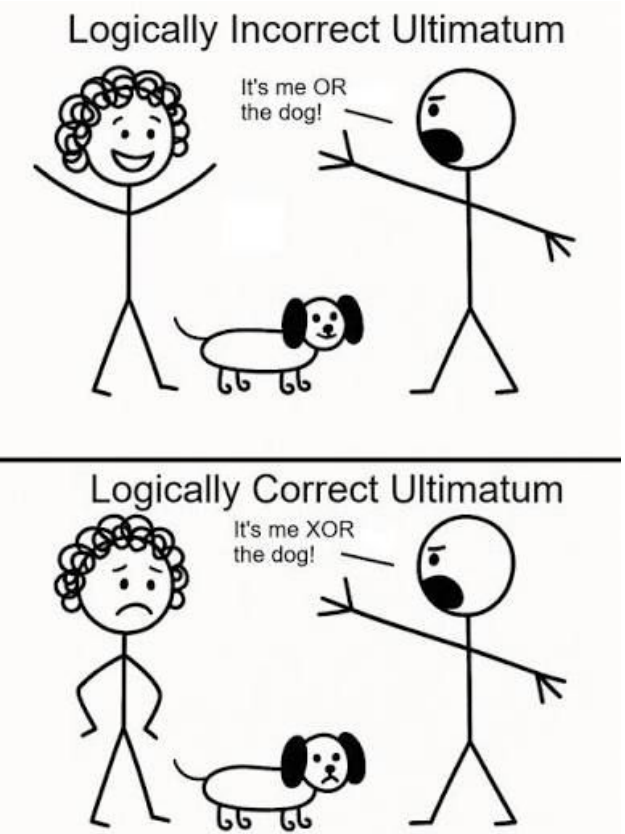
$$dp[n] = \sum_{i=0}^{n-1} dp[i]\, dp[n-1-i].$$

$$C_n = \sum_{i=0}^{n-1} C_i C_{n-1-i}.$$

עכשיו נעבור לפעולה האהובה על כולנו...

XOR!!!!

השאלה שלנו להיום:



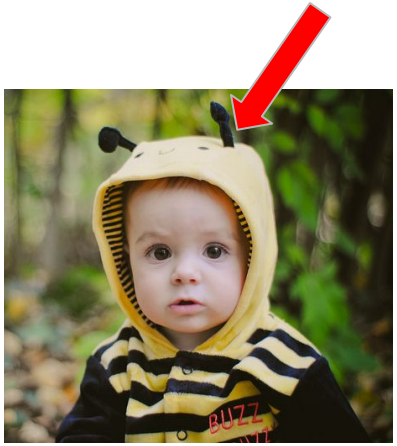
נתון לנו מערך עם n מספרים (long long) ומותר לנו לעשות:
1. לבחור תת קבוצה של מספרים
2. לעשות עליה XOR (שוק)

אנחנו רוצים לענות על השאלה הבאה:

מה הערך המקסימלי שניתן לקבל?

פתרון נאיבי וטיפשי

הוגה הפתרון
הנאיבי והטיפשי



לכל איבר יש 2 אפשרויות:
1. בפנים
2. בחוץ

לכן יש כמות ריבועית של קבוצות אפשריות, עבור כל קבוצה נעבור עליה ונחשב את ה-XOR!

$$O(2^n) \qquad or \qquad O(n * 2^n)$$

(עם חישוב xor טיפה יותר חכם)

בקיצור - נורא

אבחנה - לא אכפת לנו **מי** תת הקבוצה

יהיו 2 תתי קבוצות שונות S1,S2 של האיברים שלנו, ומתקיים: XOR(S1) = XOR(S2)

מבחינתי הן שקולות לחלוטין, אין שום חשיבות לאילו איברים השתמשתי אלא רק מה הערך שהשגתי.

רעיון: במקום להסתכל על כל תת קבוצה בנפרד, נוכל לחשוב על הקבוצה:

X = { XOR(S) | S is a subset of the input}

ואז הפכנו את הבעיה מ - "מיהו ה-XOR המקסימלי של תת הקבוצה?"
ל - "מהו האיבר המקסימלי ב-X?"

נסתכל על המספרים כוקטורים(טריוויאלי)

טריוויאלי שניתן להפוך מספר בינארי עם B ביטים לוקטור של איברים מ - $\mathbb{F}_2$

חזרנו לאלגברה א':
חיבור וקטורים - הופך להיות XOR, והכפלה בסקלר - לקחת את הוקטור או לא.

בחירת תת קבוצה נהפך להיות - צירוף לינארי של הוקטורים מעל המרחב!

לכן כל ערכי ה-XOR שניתן לייצר הם ה-span של המספרים שלנו:

$$X = \{ XOR(S) \mid S \text{ is a subset of the input} \} = span\{v1,v2,...,vn\}$$

אבחנה נוספת - לא את כל המספרים!

אם אנחנו מתעניינים רק במרחב שהם פורשים, אין סיבה לשמור וקטורים תלויים לינארית.

כלומר אנחנו נרצה למצוא בסיס לינארי שפורש את כל ערכי ה־XOR האפשריים!

עכשיו אתם בטח אומרים לעצמכם: יפה מאוד כל האבחנות האלה מר מתרגל, אבל לא הראית לנו שום דבר עד עכשיו - רק קפצת בין בעיות שקולות.

אז עכשיו סוף סוף אנחנו מגיעים לפתרון

זוכרים דירוג מטריצות? (אלגברה א עם יעל איך אפשר שלא)

איבר מוביל (pivot): בכל שורה במטריצה שאיננה שורת אפסים, הרכיב הראשון השונה מאפס ייקרא האיבר המוביל של השורה.

מטריצה מדורגת - מטריצה תקרא מטריצה מדורגת אם היא מקיימת את התנאים הבאים:

- שורות האפסים מופיעות מתחת לשורות שאינן שורות אפסים.
- האיבר המוביל בשורה ה- i נמצא משמאל לאיבר המוביל בשורה ה- i+1 (השורה שמתחתיו).

לכל שורה נסמן את הביט הדולק הגבוה ביותר בתור ה-pivot. נחזיק מערך Base בגודל מספר הביטים, כך שהאיבר במקום ה-i הוא מספר שהביט הדולק השמאלי ביותר שלו הוא i.

אלגוריתם לבדיקת שייכות לבסיס:

נניח שאנחנו רוצים להוסיף את x לבסיס. נסתכל על הביט הגבוה ביותר הדולק ב-x נסמן אותו ב-i.

אם אין לנו עדיין מספר ב base[i] אז x לא יכול להיות תלוי במספרים שכבר שמרנו.
למה? - כי לכל שאר הווקטורים בבסיס אין pivot בביט הזה. ולכן נשמור את base[i] בתור x.

אם יש לנו מספר ב base[i] אז נבצע x^=base[i].
מה קיבלנו? - ביט i נעלם, והביט הגבוה ביותר החדש של x קטן מ-i. כלומר התקדמנו לביט נמוך יותר.

למה זה מותר? נניח ש-x היה אמור להיות שייך לבסיס, אז לאחר ה-xor קיבלנו איבר חדש 'x שגם הוא עדיין צריך להיות שייך לבסיס מכיוון שהוא עדיין ב"ת לשאר הוקטורים.
(שקול לפעולה של R1=R1+R2 בדירוג מטריצות)

המשך - אלגוריתם לבדיקת שייכות לבסיס:

אחרי שביטלנו את הביט הגבוה של x, נמצא את הביט הגבוה ביותר של 'x ונבצע בדיוק את אותה הבדיקה. נעצור שמצאנו תא בבסיס בשבילו, או שהוא הפך ל-0.
אם הגענו ל-0 הוא לא שייך לבסיס, ניתן להביע אותו ע"י צירוף לינארי של כל הביטים שעשינו לו איתם XOR.

<u>הוכחה קצרה ללמה הם ב"ת:</u> ננית בשלילה שאפשר לעשות XOR של כמה איברי בסיס ולקבל 0.
נסתכל על האיבר שנבחר ל - [i]base בביט של ה-pivot שלו: לו יש 1. לכל שאר האיברים שנבחרו יש 0 בביט גבוה כזה. לכן הביט הזה יישאר 1 ב-XOR הכולל. סתירה לכך שקיבלנו 0.
ולכן איברי הבסיס בלתי תלויים.

סיבוכיות

בכל הכנסה של מספר חדש לבסיס אנחנו עוברים על הביטים שלו מהביט הגבוה ביותר לנמוך ביותר. בכל פעם שאנחנו מוצאים ביט שכבר קיים עבורו pivot בבסיס, אנחנו מבטלים אותו בעזרת XOR וממשיכים לביט נמוך יותר.

לכן כל ביט נבדק לכל היותר פעם אחת, ואם למספרים שלנו יש לכל היותר B ביטים, זמן ההכנסה של מספר אחד הוא O(B). עבור long long מתקיים B≤63, ולכן עבור n מספרים בניית כל הבסיס לוקחת O(nB), שבפועל נחשב לינארי בגודל הקלט. בנוסף, בבסיס נשמור לכל היותר איבר אחד עבור כל ביט, ולכן נשתמש ב - O(B) זיכרון.

ניזכר בבעיה המקורית

נתון לנו מערך עם n מספרים (long long) ומותר לנו לעשות:
1. לבחור תת קבוצה של מספרים
2. לעשות עליה XOR (שוק)

אנחנו רוצים לענות על השאלה הבאה:

מה הערך המקסימלי שניתן לקבל?

או באופן שקול

$$X = \{ XOR(S) \mid S \text{ is a subset of the input} \}$$
$$max(X) = ?$$

אז איך נמצא את המקסימום?

נתחיל מתוצאה השווה ל-0, ונעבור על איברי הבסיס לפי סדר יורד של ה-pivot שלהם. עבור כל איבר יש לנו שתי אפשרויות: להשאיר את התוצאה כפי שהיא, או לבצע עליה XOR עם איבר הבסיס הנוכחי.
בכל שלב נבחר בצורה גרידית באפשרות שנותנת מספר גדול יותר, וכך נבנה בהדרגה את ערך ה-XOR המקסימלי שאפשר לקבל.

```
long long ans = 0;
for(int i = 62; i >= 0; i--)
    ans = max( a: ans,  b: ans ^ basis[i]);
```

למה גרידי עובד?

בכל צעד עדיף לנו שהביט ה-i יהיה דולק, גם אם זה במחיר שכל הביטים שמתחתיו יהיו 0 כי הוא תורם יותר לסכום.
בכל צעד כל הביטים שגדולים ממני לא ישתנו בגלל דרך בניית הבסיס(בצעד ה-i כל הביטים שמשמאל ל-i הם 0 באיבר ה-i בבסיס), כלומר מעלי השגנו את התוצאה האופטימלית - וכעת אני מנסה להדליק את הביט שלי, ואם הצלחתי זה יותר טוב מכך שלא הייתי מצליח וכל הביטים מתחתיי היו 1.

לכן גרידי עובד.

קוד לבניית בסיס:

```
struct LinearBasis {
    static const int B = 63;
    long long basis[B] = {};
    void add(long long x) {
        for (int i = B - 1; i >= 0; i--) {
            if (!(x & (1LL << i)))
                continue;
            if (!basis[i]) {
                basis[i] = x;
                return;
            }
            x ^= basis[i];
        }
    }
};
void buildbase() {
    LinearBasis lb;

    for (int i = 0; i < n; i++) {
        long long x;
        cin >> x;
        lb.add(x);
    }
}
```

מה קיבלנו מהפתרון?

חדי העין ישימו לב שמהפתרון לבעיה הזו - קיבלנו פתרון לעוד 2 בעיות מגניבות כמעט בחינם:

1. כמה ערכי XOR שונים אפשר לקבל?
2. בהינתן מספר x, האם קיימת תת-קבוצה שה-XOR שלה שווה ל-x?

הפתרון של בעיה 1 הוא מיידי: נבנה בסיס פורש, ונספור את כמות האיברים בבסיס. נגיד שיש k איברים, לכן אנחנו יכולים לייצג - 2^k איברים שונים.

הפתרון של בעיה 2: נבנה בסיס פורש, ונבדוק האם x תלוי לינארית - כלומר נבצע עליו את אותו תהליך של הוספת איבר לבסיס ונראה אם אפשר להוסיף אותו. אם אפשר אז הוא לא שייך, אחרת קיימת תת קבוצה שיוצרת אותו.

הסוף